# Java OOP — Part 3.2: static, final, abstract and Nested Classes

Part 3.2 covers the three keywords that change *who owns* a member or *what may change*: `static` (belongs to the class, not to objects), `final` (cannot change) and `abstract` (incomplete, must be finished by a subclass). It ends with nested and inner classes, which rely on all three.

## 3.2.1 The static Keyword

`static` makes a member belong to the **class itself** instead of to each object. There is **one copy**, created when the class is initialized (Section 1, class loading), shared by every instance, and usable **without creating any object**.

### What can be static

| Element | Allowed? | Meaning |
|---|---|---|
| Field (variable) | Yes | One shared copy per class |
| Method | Yes | Called on the class, has no `this` |
| Initializer block | Yes | Runs once when the class is initialized |
| Nested class or interface | Yes | A nested type that needs no outer instance (3.2.4) |
| Import | Yes ( `import static` ) | Use static members by simple name |
| Top-level class | **No** | A top-level class cannot be `static` ; the idea has no meaning there |
| Constructor | **No** | A constructor always works on a new object |
| Local variable or parameter | **No** | Locals belong to one method call |

## Static fields (class variables)

```
public class Employee {
    private static int nextId = 1;           // ONE copy, shared by all
employees
    private static int headcount = 0;

    private final int id;                    // instance: one copy PER
employee
    private final String name;

    public Employee(String name) {
        this.id = nextId++;                  // every new object advances the
shared counter
        this.name = name;
        headcount++;
    }
    public static int getHeadcount() { return headcount; }
    public int getId() { return id; }
}

new Employee("Asha");
new Employee("Ravi");
System.out.println(Employee.getHeadcount());   // 2   accessed through the
class name
```

### Memory and lifetime

- Created when the class is **initialized** (first active use), not when an object is created; so it exists even if you never create an instance.
- Lives until the class is **unloaded** (usually when the application ends, or when its ClassLoader is discarded). In HotSpot the value is held with the class's `Class` object on the heap; the class's metadata sits in Metaspace.
- Gets **default values** ( `0` , `false` , `null` ) before its initializer runs (this is what causes the order traps in 3.1.6).
- Access it as `ClassName.field` . Writing `object.field` compiles but is misleading, because the object plays no role; IDEs flag it.

### Static vs instance variable

| | Instance variable | Static variable |
|---|---|---|
| Copies | One per object | One per class |
| Created | When the object is created | When the class is initialized |
| Destroyed | When the object is garbage collected | When the class is unloaded |
| Access | `object.field` | `ClassName.field` |
| Use for | Object state | Shared data, counters, constants, caches |

**Constants** are `static final`: `public static final double TAX_RATE = 0.18;` (naming: UPPER_SNAKE_CASE). Compile-time constants are inlined into callers (see 2.6 and 3.1.6).

**Mutable static state is a hazard**

- It is **global state**: any code can change it, tests influence each other, and behavior depends on call order.
- It is **not thread-safe** by default. Two threads running `headcount++` can lose updates; use `AtomicInteger` or synchronization (Multithreading notes).
- A **static collection can cause a memory leak**: `static List<Order> cache` holds every added object for the life of the class, so none of them is ever garbage collected unless you remove them.
- It is **shared across all users** in a server. A `static` field holding per-request data in a web app leaks data between users.

## Static blocks

A **static initializer block** runs **once**, when the class is initialized, in the order blocks and static field initializers appear in the source.

```
public class Countries {
    private static final Map<String, String> CODE_TO_NAME;

    static {                                          // complex setup that
cannot be one expression
        Map<String, String> m = new HashMap<>();
        m.put("IN", "India");
        m.put("US", "United States");
        CODE_TO_NAME = Collections.unmodifiableMap(m);
        System.out.println("Countries initialized");   // printed once, ever
    }

    public static String nameOf(String code) { return CODE_TO_NAME.get(code);
}
}
```

Typical uses: building constant collections, loading a native library (`System.loadLibrary("x")`), registering a JDBC driver (older style), reading configuration once.

Rules:

- Several static blocks are allowed; they run **top to bottom** along with static field initializers.
- A static block **cannot use `this` or instance members**, cannot `return`, and **cannot throw a checked exception** it does not catch.
- If it throws an unchecked exception, the class fails with `ExceptionInInitializerError` and is unusable afterwards (3.1.6).
- Keep static blocks **short and free of slow or blocking work**: they run under the class-initialization lock, so a slow block delays every thread that needs the class.
- Since Java 7 the launcher checks for `main` first, so a class with no `main` method can no longer be run just to execute its static block (an old trick).

## Static methods

```
public final class MathUtil {
    private MathUtil() { }                                          // not meant
to be instantiated
    public static int max(int a, int b)       { return a > b ? a : b; }
    public static double circleArea(double r) { return Math.PI * r * r; }
}
int m = MathUtil.max(3, 9);                                          // no
object needed
```

### Rules

- A static method has **no `this` and no `super`**, because no object is involved. It can read or call **only static members directly**. To reach instance members it needs an object reference.
- An instance method **can** use static members freely.
- It is resolved at **compile time** from the declared type of the reference (**static binding**), not at runtime from the real object.
- It **cannot be overridden**. A subclass that declares a static method with the same signature **hides** it. `@Override` on it is a compile error.
- It cannot be `abstract` (it must have a body) and, in the same way, `abstract` methods cannot be `static`.
- It can be called through a `null` reference, because the reference is ignored: `((Util) null).help()` works. (Legal but confusing.)
- It can be generic, but a **class's type parameter `T` cannot be used** in a static member; the static method declares its own: `static <T> T first(List<T> l)`.

```
class Counter {
    int value;                      // instance
    static int total;               // static

    static void bad()  { /* value++;  ERROR: instance variable from a static
context */ }
    static void ok(Counter c) { c.value++; total++; }     // needs an object
for the instance field
    void inc() { value++; total++; }                       // instance code
may use both
}
```

### Hiding is not overriding

```
class Animal {
    static String kind()   { return "animal"; }
    String sound()         { return "..."; }
}
class Dog extends Animal {
    static String kind()   { return "dog"; }               // HIDES
Animal.kind
    @Override String sound() { return "woof"; }            // OVERRIDES
Animal.sound
}

Animal a = new Dog();
System.out.println(a.kind());    // animal   static: chosen by the declared
type Animal
System.out.println(a.sound());   // woof     instance: chosen by the real
object Dog
```

This is the standard interview example for **compile-time vs runtime binding** (Part 3.3 expands it).

**Why `main` is static.** The JVM must call `main` before any object of your class exists, so it needs a method it can invoke on the class alone. (Java 25 also lets small programs use an instance `main` and the compact source file form, but the classic `public static void main(String[] args)` is still the canonical launcher.)

**Static methods in interfaces** (Java 8+) belong to the interface only, are **not inherited**, and are called as `InterfaceName.method()`.

## Static imports

A **static import** brings static members into scope by simple name:

```
import static java.lang.Math.*;
import static java.util.stream.Collectors.toList;

double r = sqrt(16) + max(2, 3);        // instead of Math.sqrt(...) and
Math.max(...)
```

Use it sparingly. Wildcard static imports hide where a name comes from. It is common and acceptable for test assertions (`assertEquals`) and well-known constants.

## Static and instance together: the class vs object view

```
class Employee                              Heap
  static nextId = 3        --->  Employee.class mirror: { nextId = 3,
headcount = 2 }
  static headcount = 2

  instance id, name        --->  object #1 { id = 1, name = "Asha" }
                                 object #2 { id = 2, name = "Ravi" }
```

The static fields exist once, with the class. Each object has only its own instance fields.

## Design guidance

| Good uses of `static` | Poor uses of `static` |
|---|---|
| Stateless helper functions (`Math.max`, `Objects.requireNonNull`) | Mutable global state, such as a static map of "current user" |
| Constants (`public static final`) | Hidden dependencies (a static call to a database or clock) that make testing hard |
| Static factory methods (`List.of`, `LocalDate.now`) | Anything that needs different behavior per instance or subclass |
| The entry point `main` | Caches with no eviction |

In frameworks such as Spring, shared services are **singleton beans** injected where needed instead of `static` utility classes with state, because injected objects can be replaced in tests and are managed by the container.

**Interview traps**

- *Can a static method access an instance variable?* Not directly; only through an object reference.
- *Can you override a static method?* No. You can hide it; the version called depends on the declared type.
- *Can a static method be abstract, or a constructor static?* No to both.
- *Is a static block executed before `main`?* Yes, if it is in the class that contains `main` (the class is initialized before `main` is called).
- *When does a static block run if the class is never used?* Never; initialization is lazy.

- *Since Java 16, inner (non-static) classes may declare static members.* Before that, only constants were allowed.

## 3.2.2 The final Keyword

`final` means **"this cannot change once set"**. What it freezes depends on where you put it: a variable's value, a method's implementation, or a class's ability to have subclasses.

| Applied to | Effect |
|---|---|
| Variable (local, field, parameter) | Can be assigned **once**; afterwards it cannot be reassigned |
| Reference variable | The **reference** cannot change; the object it points to can |
| Method | Cannot be **overridden** in a subclass |
| Class | Cannot be **extended** |

### final variables

**A final primitive local variable** holds a constant after its one assignment.

```
final int max = 100;
// max = 200;                // COMPILE ERROR: cannot assign a value to final
variable

final int limit;             // blank final: declared now, assigned later,
exactly once
if (flag) limit = 10; else limit = 20;     // OK: assigned once on every path
System.out.println(limit);
```

The compiler checks **definite assignment**: a blank final must be assigned exactly once before use, on every path. Assigning twice, or possibly zero times, is a compile error.

**Effectively final.** A local variable that is never reassigned is *effectively final* even without the keyword, and that is what lambdas and inner classes need (Section 2.5 and the Part 3.1 senior notes).

```
int base = 10;                          // never changed: effectively final
Runnable r = () -> System.out.println(base);   // allowed
// base++;                              // would make it non-effectively-final,
so the lambda would not compile
```

## final method parameters

```
static int area(final int w, final int h) {
    // w = 5;                // COMPILE ERROR: parameter cannot be reassigned
inside the method
    return w * h;
}
```

A `final` parameter only stops the method from reassigning its own copy. Because Java is pass-by-value, it never affected the caller anyway. It is a readability and safety choice, and many teams skip it because the compiler and IDE already warn about reassigned parameters.

## final reference variables

A `final` reference variable **always points to the same object**, but the object itself may still change.

```
final StringBuilder sb = new StringBuilder("hello");
sb.append(" world");                  // OK: modifying the object
// sb = new StringBuilder("x");        // ERROR: cannot re-point the reference

final int[] nums = {1, 2, 3};
nums[0] = 99;                         // OK: array elements are not final
// nums = new int[5];                 // ERROR

final List<String> names = new ArrayList<>();
names.add("Asha");                    // OK
```

**`final` is not immutability.** An immutable object cannot be changed at all; a `final` reference just cannot be re-pointed. To get a truly unchangeable collection use `List.of(...)` or `List.copyOf(...)`, and for your own classes follow the immutability recipe in Part 3.5.

## final fields

**Final instance fields** must be assigned exactly once, in one of three places: at the declaration, in an instance initializer block, or in **every** constructor.

```
class Order {
    private final long id;                          // blank final: assigned
in each constructor
    private final Instant created = Instant.now(); // assigned at declaration
    private final List<String> items;

    Order(long id) {
        this.id = id;
        this.items = new ArrayList<>();
    }
    Order(long id, List<String> items) {
        this(id);                                   // this(...) path assigns
id and items
        this.items.addAll(items);
    }
    // A constructor that left 'id' unassigned would not compile.
}
```

**Final static fields** (constants) are assigned at the declaration or in a **static block**:

```
class Settings {
    static final int TIMEOUT = 30;                 // compile-time constant
(inlined into callers)
    static final List<String> ROLES;               // runtime constant
    static { ROLES = List.of("ADMIN", "USER"); }
}
```

### Why final fields matter

- They document and **enforce** that a value never changes after construction.
- They are the basis of **immutable objects**, which are thread-safe without locks.
- The **Java Memory Model** gives final fields a special guarantee: once the constructor finishes, every thread that sees the object sees the final fields correctly set, **provided `this` did not escape** during construction (Multithreading notes and 3.1.5).
- Interface fields are implicitly `public static final`.

### Caveats

- Final instance fields can technically be modified with reflection (`setAccessible(true)`), which breaks assumptions the JIT made (it may have constant-folded the value). Recent JDKs are tightening this, so never rely on it.
- A `final` field holding a mutable object (a list, a `Date`) is still a mutable field in effect.
- A final field has **no default-value safety net**: reading it in a parent constructor through an overridden method can show `null` or `0` before assignment (3.1.6).

## final methods

A `final` method **cannot be overridden**.

```
class Report {
    public final String generate() {            // the algorithm skeleton:
subclasses must not change it
        return header() + body() + footer();
    }
    protected String header() { return "Report\n"; }
    protected String body()   { return ""; }
    protected String footer() { return "--end--"; }
}
class SalesReport extends Report {
    @Override protected String body() { return "sales data\n"; }
    // public String generate() { ... }         // COMPILE ERROR: cannot
override final method
}
```

This is the **Template Method pattern**: `generate()` fixes the sequence, and subclasses customize only the steps.

Points to know:

- Use `final` methods to protect an **invariant or algorithm**, and to make a method **safe to call from a constructor** (it cannot be overridden into surprising behavior).
- A `final` method can still be **overloaded**.
- A `private` method is not inherited and so cannot be overridden anyway; a `static` method cannot be overridden, only hidden (`final static` also forbids hiding).
- Some `Object` methods are final: `getClass()`, `notify()`, `notifyAll()`, `wait()`.
- **Performance myth:** `final` is *not* needed for the JIT to inline a method. HotSpot works out from class loading whether a method has overrides and devirtualizes it anyway. Do not add `final` "for speed".

## final classes

A `final` class **cannot be extended**.

```
public final class Money { }
// class Dollars extends Money { }       // COMPILE ERROR: cannot inherit
from final Money
```

- The JDK's `String`, `Integer` and the other wrappers, `Math` and `System` are `final`. Immutable value types must be, because a subclass could add mutable state or change behavior, breaking the guarantees callers rely on (for example a `String` used as a map key or in a security check).
- **Records** are implicitly `final`. **Enums** are implicitly final unless a constant has a body.
- All methods of a final class are effectively final.
- A class cannot be both `final` and `abstract`: abstract demands extension, final forbids it.
- **Trade-off:** final classes are safe, but they block extension and old mocking tools. Prefer **composition**, and use **`sealed`** (3.5) when you want a *limited* set of subclasses instead of none.

| Want | Use |
|---|---|
| No subclasses at all | `final class` |
| A fixed, known set of subclasses | `sealed class ... permits ...` |
| Subclassing only inside the package | package-private constructor |
| Subclassing designed and documented | a normal (non-final) class with documented extension points |

## final vs finally vs finalize

A classic interview question; three unrelated things with similar names.

| | `final` | `finally` | `finalize()` |
|---|---|---|---|
| What | A modifier (keyword) | A block in `try-catch-finally` | A method of `Object` |

| | `final` | `finally` | `finalize()` |
|---|---|---|---|
| Purpose | Prevent change, override or extension | Code that runs whether or not an exception occurred (cleanup) | Pre-collection hook of the garbage collector |
| Status | Current | Current | Deprecated for removal |

### Interview traps

- *Does `final` make an object immutable?* No. It only fixes the reference or the field.
- *Can a `final` variable be left unassigned?* A blank final may be declared without a value, but it must be assigned exactly once before use.
- *Can a final method be overloaded?* Yes. Only overriding is blocked.
- *Can a constructor be `final`?* No; constructors are not inherited, so there is nothing to protect.
- *Why is `String` final?* Security, thread safety, hash-code caching and the String pool all depend on strings never changing.
- *Is `static final` the same as a constant?* Only if its value is a compile-time constant expression of a primitive or `String` type; then it is inlined.

## 3.2.3 The abstract Keyword

`abstract` marks something as **incomplete on purpose**: a class that must not be instantiated directly, or a method that is declared but has no body. It is Java's way to say *"every real subclass must finish this"*, and it is a main tool for the pillar of **abstraction**: define **what** must be done in the parent, leave **how** to the subclasses.

### Abstract methods

An **abstract method** has a signature but **no body**; the declaration ends with a semicolon.

```
abstract double area();            // no braces, no body
protected abstract void draw(Canvas c);
```

Rules:

- It can appear **only in an abstract class** (or an interface, where methods are abstract by default).

- A **concrete** subclass must **override every inherited abstract method** with a real body. If it does not, it must itself be declared `abstract`.
- It cannot be `private` (a subclass could never see it to override it), `final` (final forbids overriding), or `static` (static methods are not overridden). It also cannot be `native`, `synchronized` or `strictfp`.
- It may be `public`, `protected` or package-private. An overriding method may **widen** the access (`protected` to `public`) but not narrow it.
- It may declare `throws` clauses, and overriding methods may throw the same exceptions, narrower ones, or none.

## Abstract classes

An **abstract class** is declared with `abstract class` and **cannot be instantiated** with `new`. It exists to be extended.

```
public abstract class Shape {
    private final String name;                // abstract classes can have
state

    protected Shape(String name) {            // and constructors (called by
subclasses via super)
        this.name = name;
    }

    public abstract double area();            // each shape computes its own
area
    public abstract double perimeter();

    public String describe() {                // concrete method, shared by
all shapes
        return name + " area=" + String.format("%.2f", area())
                    + " perimeter=" + String.format("%.2f", perimeter());
    }
}

public class Circle extends Shape {
    private final double r;
    public Circle(double r) { super("Circle"); this.r = r; }
    @Override public double area()      { return Math.PI * r * r; }
    @Override public double perimeter() { return 2 * Math.PI * r; }
}
```

```
public class Rectangle extends Shape {
    private final double w, h;
    public Rectangle(double w, double h) { super("Rectangle"); this.w = w;
this.h = h; }
    @Override public double area()      { return w * h; }
    @Override public double perimeter() { return 2 * (w + h); }
}

// Shape s = new Shape("x");        // COMPILE ERROR: Shape is abstract;
cannot be instantiated
List<Shape> shapes = List.of(new Circle(2), new Rectangle(3, 4));
for (Shape s : shapes) System.out.println(s.describe());   // each call
dispatches to the real subclass
```

The loop runs the **same code** for every shape, and each object supplies its own `area()` and `perimeter()`. That is polymorphism built on abstraction.

### What an abstract class may contain

| Member | Allowed in an abstract class? |
|---|---|
| Abstract methods | Yes (zero or more) |
| Concrete methods | Yes |
| Instance fields, static fields, constants | Yes |
| Constructors | **Yes**: run when a concrete subclass is created (via `super(...)`) |
| `final` methods | Yes (protect the algorithm) |
| Static methods and static blocks | Yes |
| Initializer blocks | Yes |
| Nested classes | Yes |

### Key rules

- If a class has **even one** abstract method, it must be declared `abstract`.

- A class **may be `abstract` with no abstract methods**. That is legal and used to forbid direct instantiation of a base class (for example `HttpServlet` has no abstract methods).
- An abstract class **cannot be `final`**: `abstract` demands extension, `final` forbids it. The compiler rejects the combination.
- A subclass of an abstract class is concrete only if it **implements all** inherited abstract methods, including ones from interfaces.
- An abstract class may **implement an interface** without implementing its methods; they simply stay abstract for subclasses.
- An abstract class **can extend** a concrete class or another abstract class, and may add or re-abstract methods.
- Abstract classes can have a **`main` method** and run like any class.

**Creating an object of an abstract type** is possible only through a concrete subclass, including an anonymous one:

```
Shape unit = new Shape("unit square") {          // anonymous subclass
supplies the missing methods
    @Override public double area()      { return 1; }
    @Override public double perimeter() { return 4; }
};
Shape ref = new Circle(1);                       // the variable's type can
be the abstract class
```

You can declare **variables, parameters and return types** of an abstract class type. Only `new Shape(...)` is forbidden.

## Constructors in abstract classes

An abstract class has constructors because it may hold state. They run as part of the constructor chain of a concrete subclass (3.1.5), parent first. Make them `protected` (or package-private), since only subclasses should call them.

```
abstract class Vehicle {
    protected final String plate;
    protected Vehicle(String plate) { this.plate = plate;
System.out.println("Vehicle ready: " + plate); }
    abstract int wheels();
}
class Bike extends Vehicle {
    Bike(String plate) { super(plate); }          // required: Vehicle has no
no-arg constructor
    int wheels() { return 2; }
}
```

The warning from 3.1.6 applies strongly here: **never call an abstract method from the abstract class's constructor**. The subclass override would run before the subclass's fields are initialized.

## Why and when to use abstract classes

- **Share code and state** among related classes that are variations of one concept (several kinds of `Shape`, `Account`, `Report`).
- **Template method:** the abstract class implements the overall algorithm with `final` steps and leaves a few steps abstract (3.2.2's `Report` example).
- **Skeleton implementations:** the JDK's `AbstractList`, `AbstractMap` and `AbstractCollection` implement most of an interface so you only write a few methods. `InputStream`, `OutputStream` and `Number` are abstract classes with shared behavior and a few methods left to subclasses.
- **Prevent misuse:** make a base class abstract when instantiating it alone would be meaningless (a plain `Shape`, a plain `Employee` when only `Manager` and `Engineer` exist).

**Abstract class or interface? (preview of 3.4)**

| | Abstract class | Interface |
|---|---|---|
| Subclass may extend | **One** class | **Many** interfaces |
| State (instance fields) | Yes | No (only constants) |
| Constructors | Yes | No |

| | Abstract class | Interface |
|---|---|---|
| Method bodies | Yes, any mix | `default`, `static`, `private` only |
| Access modifiers on members | Any | Methods are public (private helpers since 9) |
| Best for | Closely related classes sharing state or an algorithm skeleton | A capability or contract that unrelated classes can offer |

A good rule: **start with an interface** for the contract; add an **abstract class** only if you want to share state or implementation among its implementers.

### Common mistakes and interview traps

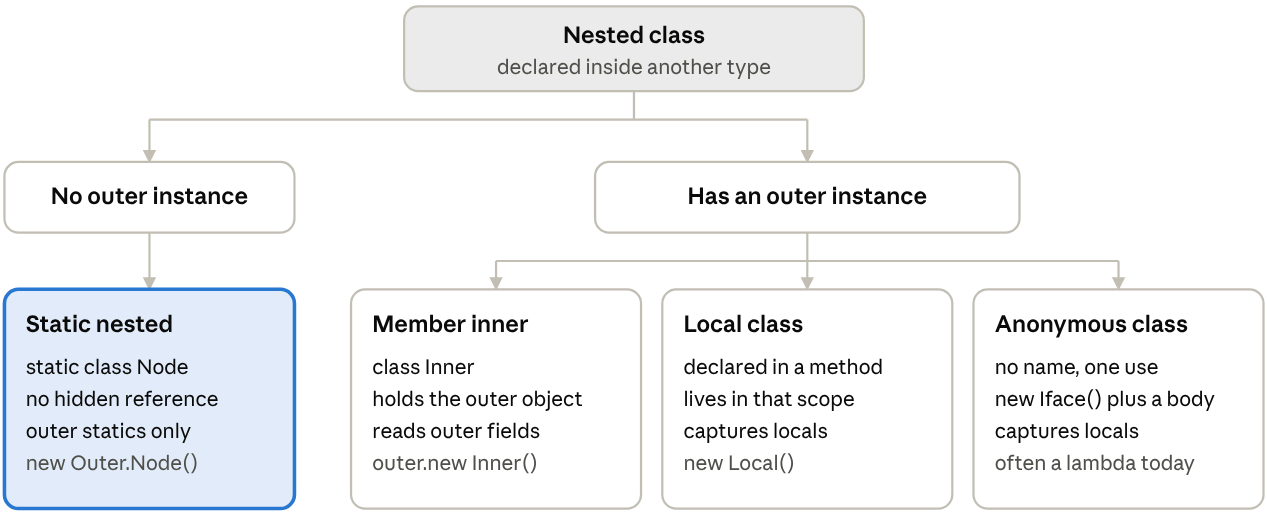
- *Can an abstract class have a constructor?* Yes, called by subclasses with `super(...)`.
- *Can you instantiate an abstract class?* Not with `new` directly; use a concrete subclass or an anonymous subclass.
- *Can a class be both `abstract` and `final`?* No. Likewise `abstract` with `private` or `static` methods is illegal.
- *Can an abstract class have no abstract methods?* Yes.
- *Can an abstract method be called?* Only through a reference to a concrete object; the call dispatches to the subclass's implementation at runtime.
- *A subclass forgets to implement one abstract method; what happens?* A compile error unless the subclass is also `abstract`.
- *Do enums use `abstract`?* An enum may declare an abstract method, and each constant then supplies a body (3.5).
- *Interface methods are abstract by default*, so you never write `abstract` on them (allowed but redundant).

## 3.2.4 Nested and Inner Classes

A **nested class** is a class declared **inside another class or interface**. The enclosing type is the **outer class**. Nesting groups a helper with the one class that uses it, hides it from everyone else, and gives it access to the outer class's private members.

**Terminology.** *Nested* means any class declared inside another type. *Static nested* means declared with `static`. *Inner* means a **non-static** nested class, which is always tied to an instance of the outer class. A **top-level** class is one that is not nested.

**Static nested has no outer object; the three inner kinds do**




Inner kinds keep a hidden reference to the enclosing object, so a long-lived inner object can keep a short-lived outer object in memory. A local or anonymous class declared inside a static method has no outer object either.

Four kinds of nested class and whether each has an outer instance

The drawing sorts the four kinds by one question: does an instance of this class need an instance of the outer class? That single fact drives memory behavior, access rules and how you create objects.

| | Static nested | Member inner | Local | Anonymous |
|---|---|---|---|---|
| Declared | In the class body, with `static` | In the class body, no `static` | Inside a method, constructor or block | Inline, in an expression |
| Has a name | Yes | Yes | Yes (visible only in its block) | No |
| Needs an outer object | **No** | **Yes** | Yes, unless declared in a static context | Yes, unless declared in a static context |
| Uses outer instance members | Only through an object reference | Directly | Directly (if instance context) | Directly (if instance context) |

| | Static nested | Member inner | Local | Anonymous |
|---|---|---|---|---|
| Uses outer private members | Yes | Yes | Yes | Yes |
| Captures local variables | No | No | Yes (effectively final) | Yes (effectively final) |
| Access modifiers | All four | All four | None | None |
| Compiled name | `Outer$Nested` | `Outer$Inner` | `Outer$1Local` | `Outer$1` |

Every nested class compiles to its **own `.class` file**, named with a `$`.

## Static nested classes

A static nested class is a normal class that **happens to live inside another class's namespace**. It has **no hidden reference** to an outer object.

```
public class Order {
    private static int created;                    // outer static (private)
    private final List<Line> lines = new ArrayList<>();

    public static class Line {                     // static nested
        private final String sku;
        private final int qty;
        public Line(String sku, int qty) {
            this.sku = sku;
            this.qty = qty;
            created++;                             // OK: outer's private
STATIC member
        }
        // lines.size();                           // ERROR: no outer Order
object to read 'lines' from
    }

    public void add(String sku, int qty) { lines.add(new Line(sku, qty)); }
}

Order.Line standalone = new Order.Line("A1", 2);   // no Order object needed
```

- It may be `public`, `protected`, package-private or `private`, and `final` or `abstract`.
- It can read the outer class's **static** members, including `private` ones. To reach an instance member it needs an explicit outer reference passed in.
- **Nested interfaces, enums and records are implicitly static.** A nested class declared in an **interface** is implicitly `public static`.
- It is the right default for nested types: no hidden reference means no hidden memory cost.

**Where you see them:** the **Builder** pattern (`Pizza.Builder`), `Map.Entry`, the private `Node` class inside `LinkedList`, `HashMap.Node`, and `@Configuration static class` helpers in Spring tests.

```
public class Pizza {
    private final String size;
    private final boolean cheese;
    private Pizza(Builder b) { this.size = b.size; this.cheese = b.cheese; }
// outer reads the builder's private fields

    public static class Builder {
        private String size = "medium";
        private boolean cheese;
        public Builder size(String s)   { this.size = s; return this; }
        public Builder cheese(boolean c){ this.cheese = c; return this; }
        public Pizza build()            { return new Pizza(this); }
    }
}
Pizza p = new Pizza.Builder().size("large").cheese(true).build();
```

## Member inner classes (non-static)

An inner class is tied to **one specific outer object**. Every inner object carries a hidden reference to the outer object that created it (named `this$0` in the bytecode).

```
public class Account {
    private double balance = 100;

    public class Statement {                          // inner class
        private final String title;
        public Statement(String title) { this.title = title; }
        public String render() {
            return title + ": " + balance;           // reads outer's private
instance field directly
        }
        public Account owner() { return Account.this; }   // explicit
reference to the outer object
    }

    public Statement newStatement(String t) { return new Statement(t); }   //
inside outer: plain 'new'
}

Account acc = new Account();
Account.Statement s = acc.new Statement("Jan");          // from outside:
outerObject.new Inner()
// new Account.Statement("Jan");                          // ERROR: no
enclosing Account instance
```

- It can use **all** outer members directly, including private instance fields and methods.
- `Outer.this` names the outer object. A same-named inner field **shadows** the outer one; use `Outer.this.x` to reach the outer field.
- Before Java 16 an inner class could declare only constant statics; since Java 16 it may declare static members freely.
- **Typical use:** iterators and views that need access to the collection's internals. `ArrayList` has a private inner class `Itr implements Iterator<E>` that reads the list's array and size directly.

**The memory leak risk.** The hidden reference keeps the **outer object alive** for as long as the inner object lives. If a long-lived structure (a listener registry, a cache, a thread) holds an inner or anonymous object, it also holds the entire outer object, even if you thought you were done with it.

```
class Screen {
    private final byte[] bigBuffer = new byte[10_000_000];
    class Listener implements Runnable { public void run() { } }      //
holds Screen.this
    void register() { Registry.INSTANCE.add(new Listener()); }       //
Registry now keeps all 10 MB reachable
}
```

Fix: make the class `static` (and pass in only what it needs), or remove the listener when done.

## Local classes

A **local class** is declared **inside a method, constructor or block**, and is visible only there.

```
static List<String> topN(List<String> words, int n) {
    class Counter {                                  // local class: no
access modifier
        int hits;
        void hit() { hits++; }
    }
    Counter c = new Counter();
    words.stream().limit(n).forEach(w -> c.hit());
    return words.subList(0, Math.min(n, words.size()));
}
```

- It cannot have an access modifier or be `static`, but can be `final` or `abstract`.
- It may use the enclosing method's **local variables and parameters** if they are **final or effectively final** (the class copies their values, the same rule as for lambdas).
- Since Java 16 you can also declare **local records, enums and interfaces**, useful for small helper data types inside one method.
- Rare in practice; lambdas and private methods usually replace it.

## Anonymous classes

An **anonymous class** declares and instantiates a class **in one expression**, with no name. It must extend one class or implement one interface.

```
List<String> names = new ArrayList<>(List.of("Ravi", "asha", "Zoya"));

Collections.sort(names, new Comparator<String>() {         // implements
Comparator, creates one instance
    @Override public int compare(String a, String b) {
        return a.compareToIgnoreCase(b);
    }
});

Thread t = new Thread() {                                    // extends
Thread, with a constructor call
    @Override public void run() { System.out.println("hello"); }
};
```

- It has **no constructor** of its own (it has no name), but it may pass arguments to the superclass constructor and may use an instance initializer block.
- It can **extend a class or implement an interface, never both**, and only one interface.
- It can capture effectively final locals and, in instance context, the outer object.
- Inside it, **`this` refers to the anonymous object**, not to the outer one.

**Anonymous class vs lambda**

| | Anonymous class | Lambda |
|---|---|---|
| Works for | Any class or interface | Only **functional interfaces** (one abstract method) |
| `this` means | The anonymous object | The enclosing object |
| State and extra methods | Yes (fields, several methods) | No |
| Compiled as | A separate `.class` file | An `invokedynamic` call site, no class file at compile time |
| Readability | Verbose | Short |

For a functional interface, prefer `names.sort(String::compareToIgnoreCase)`. Use an anonymous class only when you need state, several methods, or to extend a class.

**Anti-pattern: double-brace initialization**

```
List<String> bad = new ArrayList<>() {{ add("a"); add("b"); }};   // creates an anonymous ArrayList SUBCLASS
```

This creates a new class per use, holds a hidden reference to the outer object, breaks `equals`, and defeats serialization. Use `List.of("a", "b")` instead.

## Access rules between outer and nested

- The **outer class can see the private members of its nested classes**, and nested classes can see the outer's private members. Both share one *nest*. Before Java 11 the compiler generated hidden synthetic accessor methods to make that work; **Java 11 introduced nestmates** (JEP 181), so the JVM now allows direct private access between nest members.
- An outer class **cannot** read an inner class's members without creating an object of it.
- A nested class can be `private`, so it is completely hidden from other classes: an implementation detail.

## Choosing a kind

| Situation | Choose |
|---|---|
| A helper type used by one class, needs no outer instance (Builder, Node, Entry) | **Static nested** (default choice) |
| An object that must read the state of one specific outer object (iterator, view) | **Member inner** |
| A one-off implementation of a functional interface | **Lambda** |
| A one-off implementation with state or several methods | **Anonymous class** |
| A small helper inside one method that needs several members | **Local class** (or a private method / local record) |

## Interview traps

- *Can a static nested class access the outer's instance variables?* Only through an explicit reference to an outer object.

- *Can you create an inner class object without an outer object?* No, except inside a static context where you first create the outer: `new Outer().new Inner()`.
- *Can an inner class have static members?* Since Java 16, yes. Before that, only compile-time constants.
- *How many `.class` files does an outer class with two nested and one anonymous class produce?* Four: `Outer`, `Outer$A`, `Outer$B`, `Outer$1`.
- *Why must locals used by local and anonymous classes be effectively final?* The class holds a copy of the value; a later change to the original would make the two disagree.
- *What does `this` mean inside an anonymous class vs a lambda?* The anonymous object vs the enclosing instance.
- *Which kind can leak memory?* Inner, local and anonymous classes, through the hidden outer reference.

## Review Questions: Part 3.2

Try every question first. Part A is multiple choice, Part B is short answer, Part C is coding and prediction, and Part D is senior level. All answers and explanations are consolidated in the next section.

### Part A: Multiple Choice

**Q1.** A class has one `static int count;` field and three objects are created. How many copies of `count` exist?

- (a) 3
- (b) 1
- (c) 0
- (d) It depends on the constructor

**Q2.** Which of these can NOT be declared `static`?

- (a) A nested class
- (b) An initializer block
- (c) A top-level class
- (d) A method

**Q3.** What is printed?

```
class C { static int n; }
C a = new C();
C b = new C();
a.n = 10;
System.out.println(b.n);
```

- (a) `0`
- (b) `10`
- (c) Compile error
- (d) `null`

**Q4.** What happens?

```
class T {
    int x = 5;
    static void show() { System.out.println(x); }
}
```

- (a) Prints `5` when `show()` is called
- (b) Compile error
- (c) Prints `0`
- (d) Runtime exception

**Q5.** What is printed?

```
class Animal { static String kind() { return "animal"; } String sound() {
return "..."; } }
class Dog extends Animal { static String kind() { return "dog"; } @Override
String sound() { return "woof"; } }
Animal a = new Dog();
System.out.println(a.kind() + " " + a.sound());
```

- (a) `dog woof`
- (b) `animal woof`
- (c) `animal ...`
- (d) `dog ...`

**Q6.** What is printed?

```
class T {
    static { System.out.print("S"); }
    { System.out.print("I"); }
    T() { System.out.print("C"); }
}
new T();
new T();
```

- (a) `SICSIC`
- (b) `SICIC`
- (c) `SSIICC`
- (d) `ICSIC`

**Q7.** What happens if you put `@Override` on a `static` method that has the same signature as a parent's static method?

- (a) It overrides the parent method
- (b) Compile error
- (c) It is ignored silently
- (d) Runtime error

**Q8.** What does this do?

```
class Util { static void help() { System.out.println("ok"); } }
((Util) null).help();
```

- (a) `NullPointerException`
- (b) Prints `ok`
- (c) Compile error
- (d) Prints nothing

**Q9.** Which statement about a `static { }` initializer block is true?

- (a) It runs every time an object is created
- (b) It runs once, when the class is initialized
- (c) It runs only when `main` is present
- (d) It can use `this`

**Q10.** What happens?

```
final int x;
x = 1;
x = 2;
```

- (a) `x` is 2
- (b) `x` is 1
- (c) Compile error at `x = 2`
- (d) Compile error at `final int x;`

**Q11.** Which line fails to compile?

```
final int[] a = {1, 2};   // L1
a[0] = 9;                 // L2
a = new int[3];           // L3
```

- (a) L1
- (b) L2
- (c) L3
- (d) None

**Q12.** Which statement about a `final` method is true?

- (a) It can be overloaded
- (b) It cannot be overloaded
- (c) It cannot be inherited
- (d) It must be `static`

**Q13.** Which of these JDK classes is NOT `final`?

- (a) `String`
- (b) `Integer`
- (c) `Math`
- (d) `Object`

**Q14.** What happens with `abstract final class X { }`?

- (a) Compiles
- (b) Compile error
- (c) Compiles with a warning
- (d) Runtime error

**Q15.** Which modifier combination on a method is illegal?

- (a) `public abstract`
- (b) `protected abstract`
- (c) `abstract static`
- (d) `abstract` alone

**Q16.** Can an abstract class have a constructor?

- (a) No
- (b) Yes; it runs through `super(...)` when a subclass is created
- (c) Yes, but it must be `static`
- (d) Only if it has no abstract methods

**Q17.** If `Shape` is abstract, what does `Shape s = new Shape();` do?

- (a) Creates a `Shape`
- (b) Compile error
- (c) `InstantiationException` at runtime
- (d) Creates `null`

**Q18.** A class declares an abstract method but is not marked `abstract`. What happens?

- (a) Compiles
- (b) Compile error
- (c) Warning only
- (d) The method becomes concrete

**Q19.** A concrete class extends an abstract class and implements all but one abstract method. What happens?

- (a) Compiles; the missing method returns `null`
- (b) Compile error unless the class is also `abstract`
- (c) Runtime error when the method is called
- (d) Compiles with a warning

**Q20.** Which nested class kind has NO hidden reference to an outer object?

- (a) Member inner class
- (b) Static nested class
- (c) Anonymous class in an instance method

- (d) Local class in an instance method

**Q21.** How do you create an object of member inner class `Outer.Inner` from outside, given an `Outer o`?

- (a) `new Outer.Inner()`
- (b) `o.new Inner()`
- (c) `new o.Inner()`
- (d) `Outer.new Inner()`

**Q22.** Inside an anonymous class, `this` refers to:

- (a) The enclosing outer object
- (b) The anonymous class instance
- (c) The interface
- (d) `null`

**Q23.** Local variables used inside a local or anonymous class must be:

- (a) `static`
- (b) `volatile`
- (c) `final` or effectively final
- (d) `public`

**Q24.** `Outer` has one static nested class, one member inner class and one anonymous class. How many `.class` files are produced for `Outer` and its nested types?

- (a) 1
- (b) 2
- (c) 3
- (d) 4

**Q25.** Which statement about `final`, `finally` and `finalize` is correct?

- (a) All three are keywords
- (b) `final` is a modifier, `finally` is a block, `finalize` is a method of `Object`
- (c) `finalize` is the recommended way to release resources
- (d) `finally` runs only when no exception occurs

## Part B: Short Answer

**Q26.** List what can and cannot be `static`, and compare a static variable with an instance variable.

**Q27.** Why can a static method not be overridden? Explain hiding with an example and say which binding (compile-time or runtime) each uses.

**Q28.** Give four dangers of mutable static state.

**Q29.** What does `final` do for a variable, a method and a class? Why is `final` not the same as immutable?

**Q30.** Explain blank finals and the compiler's definite-assignment rule for final fields (instance and static).

**Q31.** Compare `final class`, `sealed class` and a class with a package-private constructor as ways of limiting subclassing.

**Q32.** State the rules for abstract methods and abstract classes, including which modifiers may not be combined with `abstract`.

**Q33.** Why do abstract classes have constructors, and what is the danger of calling an abstract method from one?

**Q34.** Describe the four kinds of nested class and when you would use each.

**Q35.** Compare an anonymous class with a lambda: `this`, capabilities, compilation.

## Part C: Coding and Prediction

**C1.** Write an abstract class `Employee` with a `name`, an abstract method `double calculatePay()` and a concrete `describe()`. Add subclasses `Salaried` (monthly salary) and `Hourly` (rate and hours). Put several in a `List<Employee>` and print the total payroll.

**C2.** Write an `Invoice` class that gets a unique, increasing id from a **thread-safe** static counter.

**C3.** Predict the output and explain:

```
class P {
    static String who() { return "P.static"; }
    String me() { return "P.me"; }
    String call() { return who() + "/" + me(); }
}
class Q extends P {
    static String who() { return "Q.static"; }
    @Override String me() { return "Q.me"; }
}
System.out.println(new Q().call());
```

**C4.** Rewrite this anonymous `Comparator` first as a lambda, then with `Comparator.comparing...`, sorting by length and then alphabetically:

```
Collections.sort(words, new Comparator<String>() {
    @Override public int compare(String a, String b) {
        int c = Integer.compare(a.length(), b.length());
        return c != 0 ? c : a.compareTo(b);
    }
});
```

**C5.** Write a generic `LinkedStack<T>` with a **private static nested** `Node<T>` class and methods `push`, `pop`, `peek` and `isEmpty`.

**C6.** This code can leak memory. Explain why and fix it:

```
class Page {
    private final byte[] images = new byte[20_000_000];
    class ClickListener implements Runnable { public void run() { /* handle
click */ } }
    void open() { GlobalEvents.register(new ClickListener()); }
}
```

**C7.** Which of these lines do not compile, and why?

```
final int a; a = 1; a++;                              // L1
abstract class A { abstract void f() { } }            // L2
class B { int x; static void m() { this.x = 1; } }    // L3
final class F { }  class G extends F { }              // L4
class H { abstract void g(); }                        // L5
```

## Part D: Senior Level

**S1.** A utility class `PriceCalculator` has `static` methods that read a static `Map<String, Double> rates` that is reloaded from the database. Explain the testing, concurrency and deployment problems, and refactor it for a Spring service.

**S2.** What exactly does the Java Memory Model guarantee for `final` fields, and what breaks the guarantee? How does it differ from `volatile`?

**S3.** "Making a method `final` makes it faster": true or false? Explain how HotSpot devirtualizes calls (class hierarchy analysis, inline caches) and when `final` still has real value.

**S4.** With default and static methods in interfaces, when is an abstract class still the better tool? Explain the "interface plus skeletal abstract class" pattern with a JDK example.

**S5.** Explain the fragile base class problem and how `final`, template methods and "design for inheritance or prohibit it" address it.

**S6.** What is the hidden `this$0` field? How did `private` access between outer and nested classes work before Java 11, and what are nestmates?

**S7.** A production service shows steadily growing heap usage, and a heap dump shows many instances of an anonymous `Runnable` holding a large outer object. Explain the mechanism and how you would find and fix it.

**S8.** How do local and anonymous classes capture local variables? Why must they be effectively final, and how does a lambda differ?

**S9.** When is an anonymous class still preferable to a lambda? Discuss `this`, state, identity, serialization and stack traces.

**S10.** Explain why a **static** method inside an **inherited** method does not behave polymorphically, using C3 as the example, and describe how you would redesign it so the subclass version is used.

# Answers: Part 3.2

## Part A: Multiple Choice

| Q | Answer | Explanation |
|---|---|---|
| 1 | (b) | A static field has one copy per class, regardless of how many objects exist. |
| 2 | (c) | A top-level class cannot be `static`; nested classes, blocks and methods can. |
| 3 | (b) `10` | Static state is shared; writing through `a` changes the one copy that `b` also reads. |
| 4 | (b) | A static method has no `this`, so it cannot read an instance variable directly. |
| 5 | (b) `animal woof` | `kind()` is static and bound to the declared type `Animal`; `sound()` is overridden and dispatched on the real `Dog`. |
| 6 | (b) `SICIC` | The static block runs once; the instance block and constructor run for each object. |
| 7 | (b) | A static method cannot be overridden, so `@Override` is a compile error. It would only hide. |
| 8 | (b) | A static method ignores the reference value, so a `null` reference works. |
| 9 | (b) | A static block runs once, at class initialization, and has no `this`. |
| 10 | (c) | A final variable can be assigned only once, so the second assignment fails to compile. |
| 11 | (c) | The final reference cannot be re-pointed (L3), but array elements can change (L2). |
| 12 | (a) | A final method can be overloaded and is inherited; it just cannot be overridden. |
| 13 | (d) | `Object` is the root class and is not final. |

| Q | Answer | Explanation |
|---|---|---|
| 14 | (b) | `abstract` needs subclasses and `final` forbids them. |
| 15 | (c) | A static method cannot be overridden, so it can never be abstract. |
| 16 | (b) | Abstract classes have constructors that run through `super(...)`. |
| 17 | (b) | An abstract class cannot be instantiated with `new`. |
| 18 | (b) | Any class containing an abstract method must itself be declared `abstract`. |
| 19 | (b) | The subclass must implement every inherited abstract method or be abstract itself. |
| 20 | (b) | A static nested class has no hidden outer reference. |
| 21 | (b) | `outerObject.new Inner()` supplies the required enclosing instance. |
| 22 | (b) | `this` is the anonymous object; the outer object is `Outer.this`. |
| 23 | (c) | The class stores a copy of the value, so the variable must never change. |
| 24 | (d) | `Outer.class`, `Outer$StaticNested.class`, `Outer$Inner.class` and `Outer$1.class`. |
| 25 | (b) | Different things: a modifier, a block in try-catch-finally, and `Object`'s (deprecated) hook. |

## Part B: Short Answer (model answers)

**Q26.** Can be static: fields, methods, initializer blocks, nested classes and interfaces, and imports (`import static`). Cannot: top-level classes, constructors, local variables and parameters. A static variable has one copy per class, is created when the class is initialized and is accessed through the class name; an instance variable has one copy per object, is created with the object, lives on the heap with it, and is accessed through a reference.

**Q27.** Overriding means the version called depends on the **runtime** class of the object. A static method belongs to the class and is called without an object, so there is no runtime object to dispatch on; the compiler binds the call to the **declared type** (static binding). A subclass static method with the same signature **hides** the parent's: `Animal a = new Dog(); a.kind()` calls `Animal.kind()` even though `a` holds a `Dog`. Instance methods are bound at runtime by the real object (dynamic binding).

**Q28.** (1) **Global state:** any code can change it, so tests influence one another and results depend on call order. (2) **Thread safety:** unsynchronized updates race (`count++` loses updates). (3) **Memory leaks:** a static collection keeps every element reachable for the life of the class. (4) **Shared across users and classloaders:** per-request data in a static field leaks between users in a server, and a static exists once per ClassLoader, so redeployed apps can hold stale copies. A fifth: hidden dependencies that cannot be mocked.

**Q29.** A `final` variable can be assigned once; a `final` method cannot be overridden; a `final` class cannot be extended. `final` freezes the *binding*, not the *object*: a `final List` can still have elements added. An immutable object is one whose state cannot change after construction, which needs all-final fields **and** no mutable internals exposed (Part 3.5).

**Q30.** A blank final is a final variable declared without a value. The compiler requires **definite assignment**: it must be assigned exactly once on every path before use. A final instance field must be assigned in its declaration, in an instance initializer, or in **every** constructor. A static final field must be assigned in its declaration or in a static block. Assigning twice, or leaving a path that does not assign, is a compile error.

**Q31.** `final class`: no subclasses at all. `sealed class`: only the subclasses named in `permits`, so the hierarchy is closed but known (enabling exhaustive `switch`). Package-private constructor: only classes in the same package can extend it, a convention-level limit that is weaker than the other two. Choose `final` for value types, `sealed` for fixed families of variants, and a package-private constructor for internal hierarchies.

**Q32.** An abstract method has no body and ends with `;`; it may appear only in an abstract class or interface; a concrete subclass must implement it; it cannot be `private`, `final`, `static`, `native`, `synchronized` or `strictfp`. An abstract class cannot be instantiated; it may have fields, constructors, concrete and static methods; a class with an abstract method must be abstract; an abstract class cannot be `final`; and a class may be abstract with no abstract methods.

**Q33.** The abstract class may hold state that must be initialized, so its constructor runs through `super(...)` when a concrete subclass is created. Calling an abstract (or any overridable) method from it is dangerous: the override runs before the subclass's constructor and field initializers have run, so it sees default values.

**Q34.** *Static nested:* a helper type that needs no outer object (Builder, Node, Entry); the default choice. *Member inner:* an object that must read one specific outer object's state (an iterator or view). *Local:* a small named helper used only inside one method (rare; a private method or local record is usually better). *Anonymous:* a one-off implementation of a class or interface, when a lambda cannot be used because it needs state, several methods or a superclass.

**Q35.** An anonymous class can implement any interface or extend any class, may have fields and several methods, and `this` refers to the anonymous object; it compiles to its own `.class` file. A lambda works only for functional interfaces, has no state of its own, `this` means the enclosing instance, and it compiles to an `invokedynamic` call site with no class file at compile time. Prefer lambdas when they fit.

## Part C: Coding and Prediction (model solutions)

### C1. Payroll

```
abstract class Employee {
    private final String name;
    protected Employee(String name) { this.name = name; }
    public abstract double calculatePay();
    public String describe() { return name + " earns " +
String.format("%.2f", calculatePay()); }
}
class Salaried extends Employee {
    private final double monthly;
    Salaried(String name, double monthly) { super(name); this.monthly =
monthly; }
    @Override public double calculatePay() { return monthly; }
}
class Hourly extends Employee {
    private final double rate, hours;
    Hourly(String name, double rate, double hours) { super(name); this.rate =
rate; this.hours = hours; }
    @Override public double calculatePay() { return rate * hours; }
}
```

```
List<Employee> staff = List.of(new Salaried("Asha", 60_000), new
Hourly("Ravi", 500, 120));
staff.forEach(e -> System.out.println(e.describe()));
double total = staff.stream().mapToDouble(Employee::calculatePay).sum();   //
120000.0
```

### C2. Thread-safe id generator

```
public class Invoice {
    private static final AtomicLong SEQUENCE = new AtomicLong();     // one
shared, atomic counter
    private final long id = SEQUENCE.incrementAndGet();                 //
each object gets its own final id
    public long getId() { return id; }
}
```

A plain `static long counter; id = ++counter;` loses updates when two threads create invoices at once. `AtomicLong` (a CAS loop) fixes that without locking.

### C3. Output: `P.static/Q.me`

`call()` is declared in `P`, so the call `who()` inside it is compiled as a **static call to `P.who()`**. Static methods are bound by compile-time type and never dispatched on the object, so `Q.who()` is never considered. `me()` is an instance method, so it is dispatched on the real object (`Q`).

### C4. Comparator rewrites

```
// lambda
words.sort((a, b) -> {
    int c = Integer.compare(a.length(), b.length());
    return c != 0 ? c : a.compareTo(b);
});

// comparator combinators (clearest)
words.sort(Comparator.comparingInt(String::length).thenComparing(Comparator.n
aturalOrder()));
```

### C5. LinkedStack

```
public class LinkedStack<T> {
    private static class Node<T> {                 // static: needs no outer
object; declares its OWN <T>
        final T value;
        final Node<T> next;
        Node(T value, Node<T> next) { this.value = value; this.next = next; }
    }

    private Node<T> head;
    private int size;

    public void push(T v) { head = new Node<>(v, head); size++; }
    public T pop() {
        if (head == null) throw new NoSuchElementException("stack is empty");
        T v = head.value;
        head = head.next;
        size--;
        return v;
    }
    public T peek() {
        if (head == null) throw new NoSuchElementException("stack is empty");
        return head.value;
    }
    public boolean isEmpty() { return head == null; }
    public int size() { return size; }
}
```

A static nested class cannot use the outer class's type parameter `T`, so `Node` declares its own.

### C6. The leak and the fix

`ClickListener` is a member inner class, so every instance holds `Page.this`. The global registry keeps the listener alive, which keeps the whole `Page` and its 20 MB `images` array reachable even after the page is closed.

```
class Page {
    private final byte[] images = new byte[20_000_000];
    private final ClickListener listener = new ClickListener();

    static class ClickListener implements Runnable {           // static: no
hidden Page reference
        @Override public void run() { /* handle click, using only what it
needs */ }
    }
    void open()  { GlobalEvents.register(listener); }
    void close() { GlobalEvents.unregister(listener); }       // always
remove long-lived registrations
}
```

**C7. Compile errors: all five lines fail**

- L1: `a++` modifies a final variable that was already assigned.
- L2: an abstract method cannot have a body.
- L3: `this` does not exist in a static method.
- L4: `G` cannot extend the final class `F`.
- L5: a class with an abstract method must be declared `abstract`.

## Part D: Senior Level (model answers)

**S1.** *Testing:* static state is global, so tests share and contaminate it, cannot run in parallel, and you cannot substitute the database without reflection tricks. *Concurrency:* reloading a mutable `Map` while other threads read it risks torn reads and `ConcurrentModificationException`. *Deployment:* a static exists once per ClassLoader (duplicated in app servers), once per JVM (stale across several service instances), and has no managed lifecycle. Refactor to an injected service that holds an **immutable snapshot** swapped atomically:

```
@Service
public class PriceCalculator {
    private final RateRepository repo;
    private volatile Map<String, Double> rates = Map.of();       // immutable
snapshot
    public PriceCalculator(RateRepository repo) { this.repo = repo; }

    @Scheduled(fixedDelay = 60_000)
    void refresh() { rates = Map.copyOf(repo.loadAll()); }          // build
fully, then publish with one write

    public double price(String sku, int qty) { return rates.getOrDefault(sku,
0.0) * qty; }
}
```

In tests, pass a fake `RateRepository`. Truly pure helpers (rounding, formatting) can remain `static`.

**S2.** The JLS (17.5) says that when a constructor finishes, the object's `final` fields are "frozen": **any thread that obtains a reference to the object, even through a data race, sees the final fields' constructor-time values**, and sees objects reachable only through those final fields (a final list's contents added during the constructor) as they were at that moment. It does **not** cover non-final fields, later mutations of referenced objects, or an object whose `this` escaped before the constructor ended. Reflection can also change final fields. `volatile` is different: it gives visibility and ordering for **repeated** writes and reads of a variable by establishing happens-before from each write to later reads; final is a **one-time** guarantee at construction and needs no happens-before for readers.

**S3.** False. HotSpot uses **class hierarchy analysis**: if a method currently has a single implementation among the loaded classes, the JIT inlines it with a recorded dependency and deoptimizes if a new subclass appears. At call sites it also uses profile-based **inline caches** (monomorphic and bimorphic) with a cheap type check, and falls back to vtable or itable dispatch only for megamorphic sites. So `final` adds little to the generated code. Its real value is **design**: preventing unsafe overriding, enabling safe immutability, the final-field memory-model guarantee, `static final` constant folding and `javac` constant inlining.

**S4.** Choose an abstract class when the types need **shared state**, constructors that establish invariants, **non-public** members (protected hooks), or `final` methods that protect an algorithm. Interface `default` methods cannot touch state, cannot be `final` (implementers can override them), and are always public. The **skeletal implementation**

pattern combines both: the interface defines the type and the abstract class implements most of it from a few primitives. `List` plus `AbstractList` is the standard example: implement `get(int)` and `size()` and you inherit `iterator`, `indexOf`, `equals`, `hashCode` and more. The same applies to `Map` / `AbstractMap` and `Collection` / `AbstractCollection`. A class that must extend something else implements the interface and delegates to a private inner class that extends the skeleton.

**S5.** The **fragile base class problem**: changing a parent's *implementation* (not its API) breaks subclasses that relied on it. Classic case: a subclass of `HashSet` overrides `add` and `addAll` to count insertions, but `HashSet.addAll` internally calls `add`, so elements are counted twice. Remedies: document self-use of overridable methods; use **template methods** where the parent has a `final` skeleton and narrow `protected` hooks; prefer **composition and forwarding** (a wrapper holding a `Set`) to inheritance; and follow "**design and document for inheritance, or else prohibit it**" with `final` or `sealed`.

**S6.** `this$0` is a synthetic field in an inner class holding the outer instance; the constructor receives the outer object as a hidden first parameter. Because a nested class compiles to a **separate class**, the JVM saw its access to the outer's `private` members as illegal. Before Java 11, `javac` generated hidden static package-private **accessor methods** (such as `access$000`) in the outer class. **Nestmates** (JEP 181, Java 11) add `NestHost` and `NestMembers` attributes so the JVM itself permits private access among members of one nest, removing the synthetic accessors, shrinking bytecode and fixing odd reflection results.

**S7.** An anonymous (or inner) class created in an instance method captures `this$0`, so the registered `Runnable` keeps the whole outer object reachable. If registrations pile up and are never removed, the outer objects accumulate. Find it with `-XX:+HeapDumpOnOutOfMemoryError` or `jcmd <pid> GC.heap_dump`, open the dump in Eclipse MAT, look at the **histogram and dominator tree** for classes such as `Outer$1`, and use **path to GC roots** to see the registry or executor holding them (through `this$0` or `val$...` fields). A `GC.class_histogram` taken over time shows the growth. Fixes: unregister at the end of the lifecycle; use a **static nested class** or a lambda that does not capture `this`; capture only the needed values; or use weak references for listeners. (A lambda that uses `this` or an instance member retains the outer object too.)

**S8.** The compiler copies each captured local into a **synthetic field** (named `val$x`) of the local or anonymous class and passes the value to its constructor. Methods of the class read the copy. Since the class can outlive the method call, and run on other threads, the original and the copy could disagree if the variable changed, so the language requires the

variable to be **final or effectively final**. A lambda also captures by value under the same rule, but compiles to an `invokedynamic` call site whose captured values are passed to the generated function object; it has no outer reference unless the body uses `this` or an instance member, and its `this` means the enclosing instance.

**S9.** Anonymous classes are still right when you must **extend a class** (including with constructor arguments), implement an interface with **several abstract methods**, keep **their own fields**, or need `this` to refer to the object itself (for example to unregister it). Differences: `this` (anonymous object vs enclosing instance); state (yes vs none); identity (a capturing lambda creates a new instance per evaluation, a non-capturing one may be reused; neither is guaranteed); serialization (lambda serialization is fragile; anonymous classes serialize like any class but capture the outer); stack traces (named `Outer$1` vs synthetic `lambda$method$0`); startup (an anonymous class is a class to load, a lambda is linked at first use via `invokedynamic`).

**S10.** Inside `P.call()` the expression `who()` is compiled to `invokestatic P.who`. A static call is resolved at **compile time** against the class where the call appears; the actual runtime type of the receiver never takes part, so `Q.who()` can never replace it. Redesign so the varying behavior is an **instance method**, which is dispatched at runtime:

```
class P {
    String who() { return "P.who"; }                // instance,
overridable
    String me()  { return "P.me"; }
    String call() { return who() + "/" + me(); }
}
class Q extends P {
    @Override String who() { return "Q.who"; }
    @Override String me()  { return "Q.me"; }
}
// new Q().call()  ->  Q.who/Q.me
```

If the value is only a per-class constant, pass it to the parent constructor or use an overridable `protected` method instead of a static one.

---

**End of Part 3.2.** Next is **Part 3.3: inheritance and polymorphism** (IS-A, HAS-A, aggregation and composition, the inheritance types, overloading vs overriding, dynamic dispatch, shallow vs deep copy). Tell me when to start, or flag anything here you want expanded first.